



Paradigmas de Programación (EIF-400) Caso de FP-OOP en Java Parte-A

CARLOS LORÍA-SÁENZ LORACARLOS@GMAIL.COM

AGOSTO-SETIEMBRE 2025

EIF/UNA

Objetivos

2

- ▶ Revisar el paradigma de FP en el caso de Java
- ▶ Conocer las formas de combinar paradigmas OOP y FP en el caso de estudio:
 - ▶ OOP: datos
 - ▶ FP: control basado en expresiones no estatutos
- ▶ Evaluar el caso de FP en el contexto de tipificación estático

Objetivos Específicos

3

- ▶ Repaso principios FP
- ▶ Lambdas como objetos
- ▶ Streams
- ▶ Combinadores básicos en Colecciones
- ▶ SAM e Interfaces Funcionales
- ▶ Métodos default
- ▶ Clausuras y Scope
- ▶ Los colectores y funciones de agregación en la segunda parte

Resumen: FP

4

- ▶ Función es objeto de “*primer piso*”: abstrae computación
- ▶ Programar = Razonar/Combinar funciones
 - ▶ Combinadores de control explícitos
 - ▶ Recursión
 - ▶ Inmutabilidad maximizada (data types)
- ▶ Evaluar = simplificar expresiones
- ▶ Minimizar efectos-secundarios (SE):
 - ▶ Aumentar transparencia referencial
- ▶ Maximizar Literalidad (Knuth)
- ▶ Maximizar DRY
- ▶ Promover Estilo fluido

- ▶ El **objeto** es por un lado **dato** (sustantivo)
- ▶ Pero puede encapsular **control** (métodos)
- ▶ Control = verbos de computación
- ▶ En “imperativo” el **controlador** es la máquina: un objeto implícito:
 - ▶ ¿A quién le pertenece `while` por ejemplo?
- ▶ EN OOP-FP movemos patrones de control de la máquina a los objetos usando combinadores
- ▶ Los métodos son **funciones** del objeto
- ▶ La función es la tarea a realizar

Pipelines (líneas de producción)

- ▶ Un programa es como un flujo de trabajos (pipeline)
- ▶ Lo inicia un objeto
- ▶ “.” es una composición de funciones
- ▶ Cada paso produce un resultado (objeto de transferencia) que es la entrada del siguiente paso
- ▶ Se prefiere no mutar durante el flujo
- ▶ Mutar pueda crear prob

Lambda

7

- ▶ Origina en el cálculo Lambda de Church
- ▶ Modela una Función: Por ejemplo:
 - ▶ $add: Nat \rightarrow Nat$
 - ▶ $add : x \rightarrow x + 666$
- ▶ Es un objeto que encapsula código
- ▶ Ejemplo (jshell)

```
JAVA:jshell
| Welcome to JShell -- Version 24.0.1
| For an introduction type: /help intro
```

```
jshell> Function<Integer, Integer> add_one = x -> x + 1
add_one ==> $Lambda/0x0000019a9800dd50@6659c656
```

```
jshell> add_one.
andThen(      apply(      compose(      equals(      getClass()      hashCode()      notify()
notifyAll()   toString()   wait(
jshell> add_one.|
```

Análogo en otros lenguajes

8

► JS:

```
> let add = x => x + 666  
undefined  
> |
```

► Python:

```
>>> add = lambda x: x + 666  
>>> |
```


Un objeto “normal”

- ▶ Ejemplo: un objeto de tipo String
- ▶ Tiene **métodos**
- ▶ Es **asignable**
- ▶ Se le puede pasar de **parámetro** a un método

```
jshell> String s = "Hello!"  
s ==> "Hello!"
```

```
jshell> s.
```

charAt()	chars()	codePointAt()	codePointBefore()	codePointCount()
codePoints()	compareTo()	compareToIgnoreCase()	concat()	contains()
contentEquals()	describeConstable()	endsWith()	equals()	equalsIgnoreCase()
formatted()	getBytes()	getChars()	getClass()	hashCode()
indent()	indexOf()	intern()	isBlank()	isEmpty()
lastIndexOf()	length()	lines()	matches()	notify()
notifyAll()	offsetByCodePoints()	regionMatches()	repeat()	replace()
replaceAll()	replaceFirst()	resolveConstantDesc()	split()	startsWith()
strip()	stripIndent()	stripLeading()	stripTrailing()	subSequence()
substring()	toArray()	toLowerCase()	toString()	toUpperCase()
transform()	translateEscapes()	trim()	wait()	

```
jshell> s.|
```

Lambda: objeto especial

10

- ▶ Tiene **métodos**
- ▶ Es **invocable** (aplicable, llamable)
- ▶ Es **asignable**
- ▶ Puede ser **parámetro** de un método u otra lambda

```
jshell> add.  
andThen(      apply(      compose(      equals(      getClass()      hashCode()      notify()      notifyAll()  
toString()    wait()  
jshell> add.|
```

```
jshell> add.apply(10)  
$2 ==> 676
```

Llamada/nvocación/apply

```
jshell> var sumar = add  
sumar ==> $Lambda$20/0x00000000800c0e800@e2144e4
```

asignación

```
jshell> sumar.apply(10)  
$4 ==> 676
```

Un método la llama

```
jshell> int sumarComoMetodo(int x){ return add.apply(x) + 20;}  
| created method sumarComoMetodo(int)
```

Lambda como parámetro

11

```
jshell> Integer someMethod(Function<Integer, Integer> lambda, Integer number){  
...>     return lambda.apply(number);  
...> }  
| created method someMethod(Function<Integer, Integer>,Integer)  
  
jshell> someMethod(add, 10)  
$11 ==> 676  
  
jshell> |
```

Lambda parámetro

12

```
jshell> BiFunction<Function<Integer, Integer>, Integer, Integer> anotherLambda = (lambda, num) -> lambda.apply(num)
anotherLambda ==> $Lambda$27/0x00000000800c1280@64a294a6

jshell> anotherLambda.apply(add, 10)
$14 ==> 676

jshell> |
```

BiFunction<S, T, R> tipo de lambda de dos argumentos
tipo S y T, retorna algo de tipo R

En el ejemplo

S = Function<Integer, Integer>

T = Integer

R = Integer

Regla de Java

13

- ▶ El compilador le debe poder asignar un tipo a cada objeto
- ▶ Las lambdas son **objetos**
- ▶ Por ende, tienen que tener un tipo
 - ▶ Hay **tipos disponibles** en el JDK
 - ▶ Usando interfaces se pueden crear nuevos tipos (**interfaces funcionales**)

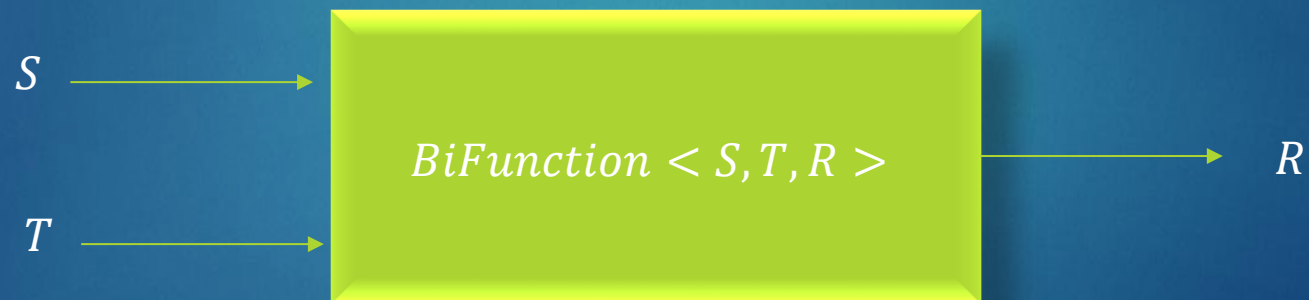
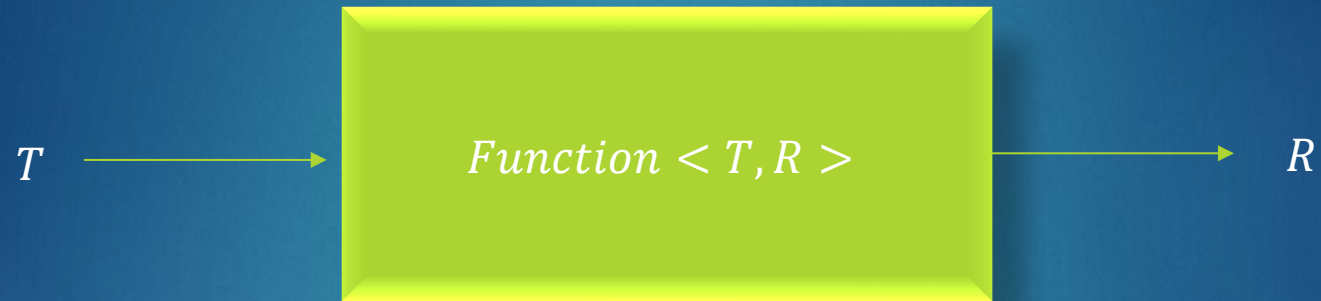
Tipos Disponibles

14

- ▶ Ver [acá](#) (paquete [java.Lang.function](#))
- ▶ Ejemplos comunes
- ▶ `Function`
- ▶ `BiFunction`
- ▶ `UnaryOperator`
- ▶ `BinaryOperator`
- ▶ `Supplier`
- ▶ `Consumer`
- ▶ `Predicate`

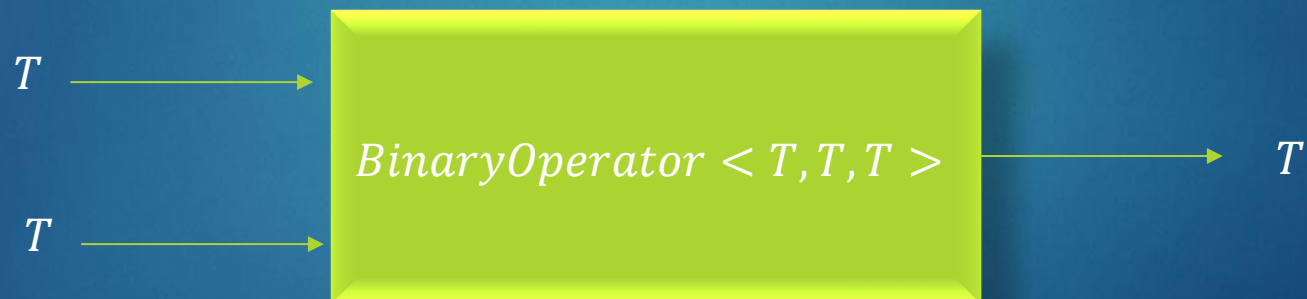
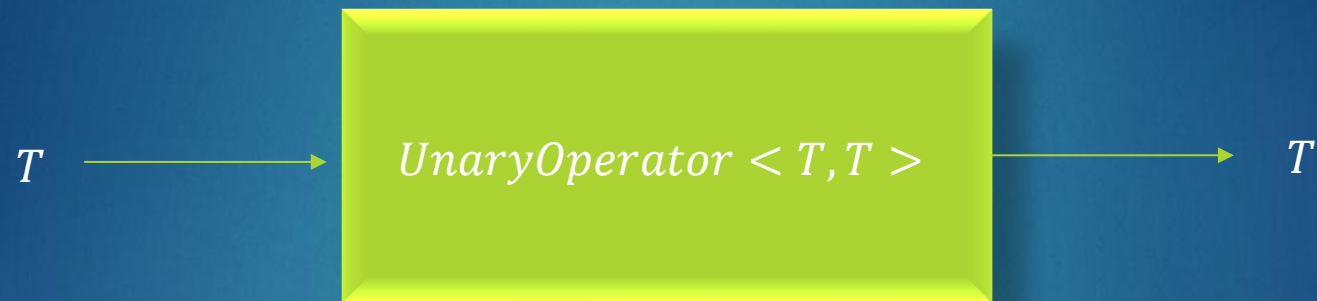
Tipos comunes: Función

15



Tipos comunes: Operador

16



Tipos comunes: Predicado

17



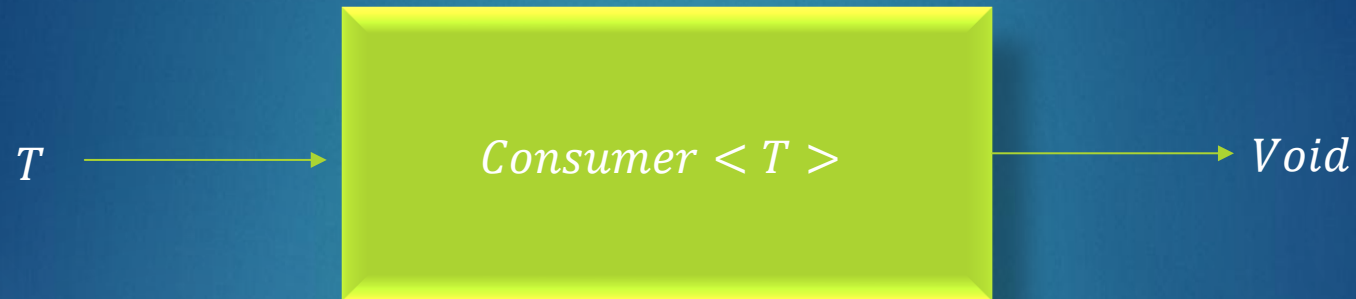
Tipos comunes: Supplier

18



Tipos comunes: Consumer

19



Especializaciones

20

- ▶ Hay **especializaciones** de varios de los tipos anteriores: Ejemplos
 - ▶ `IntFunction<R>`
 - ▶ `IntConsumer`
 - ▶ `IntPredicate`
 - ▶ `IntToDoubleFunction`
 - ▶ ...

Interfaz Funcional

21

- ▶ Una interfaz SAM (single abstract Method) es una que **tiene exactamente un método abstracto**
- ▶ Toda SAM es entendida por el compilador como interfaz funcional
- ▶ Hay una anotación `@FunctionalInterface` que se recomienda ponerle a una SAM para que el compilador confirme que es realmente funcional
- ▶ **Regla:** en cualquier “lugar” donde se esperan objetos que cumplan una Interfaz Funcional puede aceptar una lambda siempre que cumpla con la signatura del método abstracto de la interfaz

Ejemplo

22

- ▶ `java.lang.Runnable` es una interfaz funcional.
- ▶ Su método abstracto es `void run()`

Se espera un Runnable

```
jshell> void startRunning(Runnable r){ new Thread(r).start();}  
| created method startRunning(Runnable)
```

```
jshell> startRunning(() -> {for (int i=0;i < 10; i++) System.out.println("Hola "+ i);})
```

```
jshell> Hola 0
```

```
Hola 1
```

```
Hola 2
```

```
Hola 3
```

```
Hola 4
```

```
Hola 5
```

```
Hola 6
```

```
Hola 7
```

```
Hola 8
```

```
Hola 9
```

```
jshell> |
```

La lambda pasa a ser el run
del Runnable r

Interfaces y su método abstracto

23

Interfaz Funcional	Método Abstracto
<code>Function<T, R></code>	<code>R apply(T x)</code>
<code>Predicate<T></code>	<code>Boolean test(T x)</code>
<code>Supplier<R></code>	<code>R get()</code>
<code>Consumer<T></code>	<code>void accept(T x)</code>

Métodos default

24

- ▶ Las interfaces pueden en JDK+8 tener implementaciones default
- ▶ No pueden tener estado
- ▶ Permiten herencia múltiple (de conducta no de estado)

- ▶ Método heredado en conflicto si es implementado en dos interfaces o en una superclase y una o más interfaces
- ▶ 1. Clase tiene prioridad sobre cualquier interfaz con que entre en conflicto
- ▶ 2. Entre dos interfaces con conflicto la más específica gana
- ▶ 3. Si hay conflicto y no aplican ni 1 ni 2 aplican se debe declarar abstracto el método en conflicto o implementarlo

Ejemplo

26

Problema

```
jshell> class P{  
...>     public void goo(){}  
...> }  
| created class P
```

```
jshell> interface IA{  
...>     default void foo(){}  
...>     default void goo(){}  
...> }  
| created interface IA
```

```
jshell> interface IB{  
...>     default void foo(){}  
...>     default void goo(){}  
...> }  
| created interface IB
```

```
jshell> interface IC extends IB {  
...>     default void foo(){}  
...>     default void hoo(){}  
...> }  
| created interface IC
```

```
jshell> class C1 extends P implements IA, IB, IC{  
| Error:  
| types IA and IC are incompatible;  
| class C1 inherits unrelated defaults for foo() from types IA and IC  
| class C1 extends P implements IA, IB, IC{  
| ^-----^
```

Una posible solución: Que C1 implemente foo

```
jshell> class C1 extends P implements IA, IB, IC{  
...>     public void foo(){}  
...> }  
| created class C1
```

Otra posible solución: C1 declare foo y sea abstracta

```
jshell> abstract class C1 extends P implements IA, IB, IC{  
...>     abstract public void foo();  
...> }  
| created class C1
```


Inferencia de tipos

- ▶ El compilador puede en muchos casos **inferir** el tipo de un objeto en el contexto donde se menciona al objeto.
- ▶ Inferir tipo: averiguar cuál es un tipo que puede servir
- ▶ A veces hay que ayudarlo porque la inferencia no es la esperada por el programador. O no es inferible

```
jshell> Boolean usePredicate(Predicate<Integer> oper, Integer value){ return oper.test(value);}
| created method usePredicate(Predicate<Integer>,Integer)
```

```
jshell> usePredicate( $x \rightarrow x > 0$ , 666)
$22 ==> true
```

```
jshell> |
```

Se infiere que $x \rightarrow x > 0$ es de tipo Predicate<Integer>

var no permite inferencia

28

```
jshell> var add = (Integer x) -> x + 666
| Error:
| cannot infer type for local variable add
| (lambda expression needs an explicit target-type)
| var add = (Integer x) -> x + 666;
| ^-----^

jshell> |
```

```
jshell> var add = (UnaryOperator<Integer>)((Integer x) -> x + 666)
add ==> $Lambda$31/0x0000000800c14600@759ebb3d

jshell> |
```

Casting explícito funciona, pero es “feo”

Lambda: No es método

29

- ▶ En Java:
 - ▶ Un método solo existe en una clase o instancia de una clase. **No tiene identidad** por sí mismo
 - ▶ Una lambda existe independiente de una clase. **Tiene identidad**

```
jshell> class A{
...>     Integer add(Integer x){return x + 666;}}
...> }
| created class A

jshell> var a = new A()
a ==> A@593634ad

jshell> a.add(10)
$9 ==> 676

jshell> |
```

A::add le pertenece a A

A::add se invoca como parte de un A

“Igualdad” de Lambdas

30

- Operador `==` con lambdas solo compara como referencias nunca por “contenido” o “valor”

```
jshell> Function<Integer, Integer> add1 = x -> x + 10
add1 ==> $Lambda$28/0x00000000800c12e28@1a93a7ca

jshell> Function<Integer, Integer> add2 = x -> x + 10
add2 ==> $Lambda$29/0x00000000800c13260@61e717c2

jshell> add1 == add2
$17 ==> false

jshell> |
```

Uso con colecciones

- ▶ Patrones (combinadores básicos)
 - ▶ `map`
 - ▶ `filter`
 - ▶ `reduce`
 - ▶ `forEach`
- ▶ `map`: Convertir una colección A en otra colección B usando una transformación F
- ▶ `filter`: sacar todos los elementos de una colección A que cumple una condición F
- ▶ `reduce`: Tomar una colección A y un reductor R y reducir A a un único valor v .
- ▶ `forEach`: un `reduce` que retorna `void` (no retorna nada)

Ejemplos

32

▶ **map**

- ▶ Una lista de persona retornar la lista de sus nombres
- ▶ Una lista de empleados aumentarles a todos el salario

▶ **filter**

- ▶ De una lista de personas obtener la lista de mujeres mayores de edad
- ▶ De una lista de equipos los que tienen más de 10 campeonatos

▶ **reduce:**

- ▶ Sumar las edades de las personas de una lista
- ▶ Encontrar la edad de la persona de máxima edad de una lista de personas

▶ **forEach:**

- ▶ Imprimir los nombres de las personas de una lista

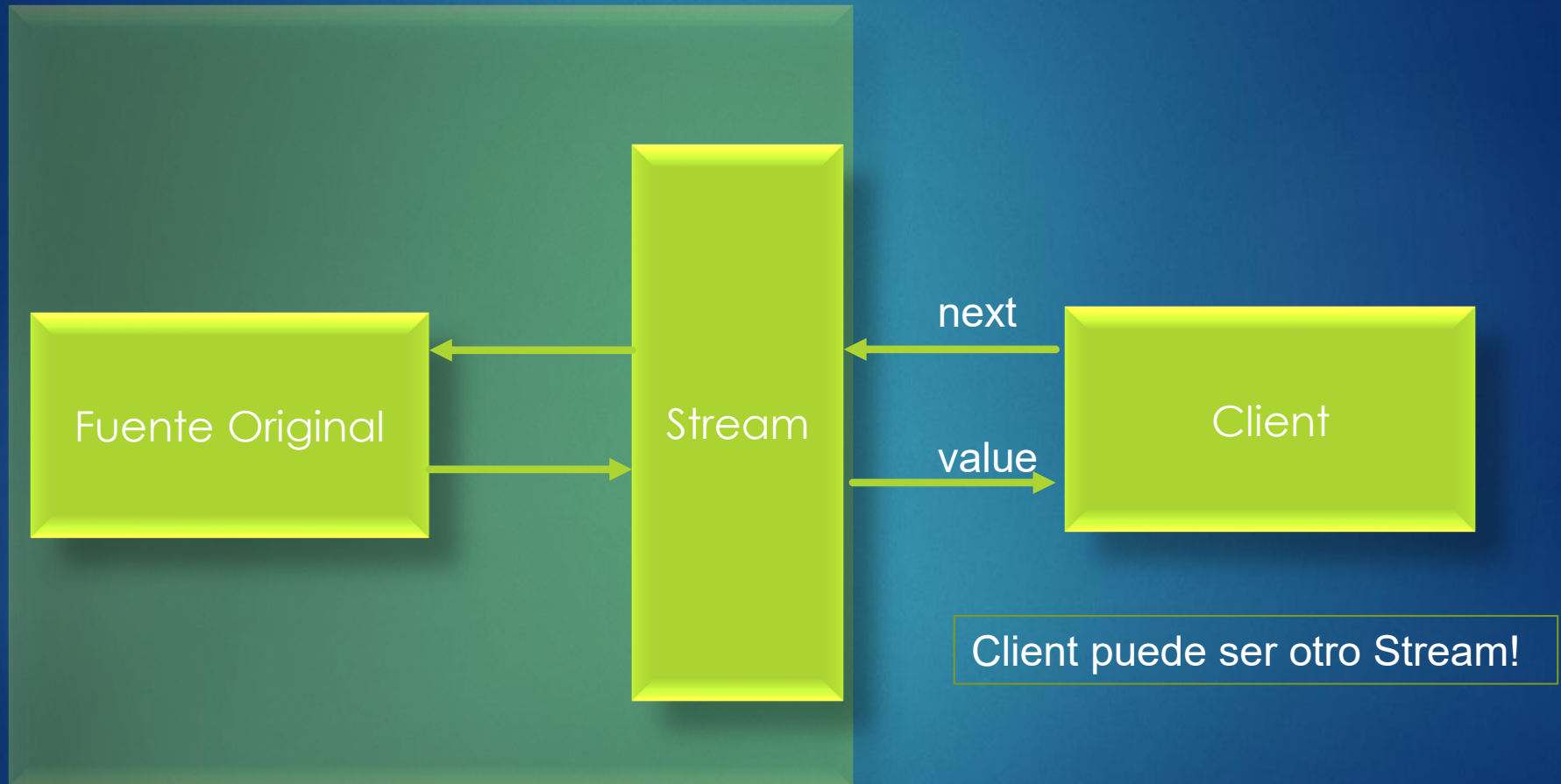
Stream

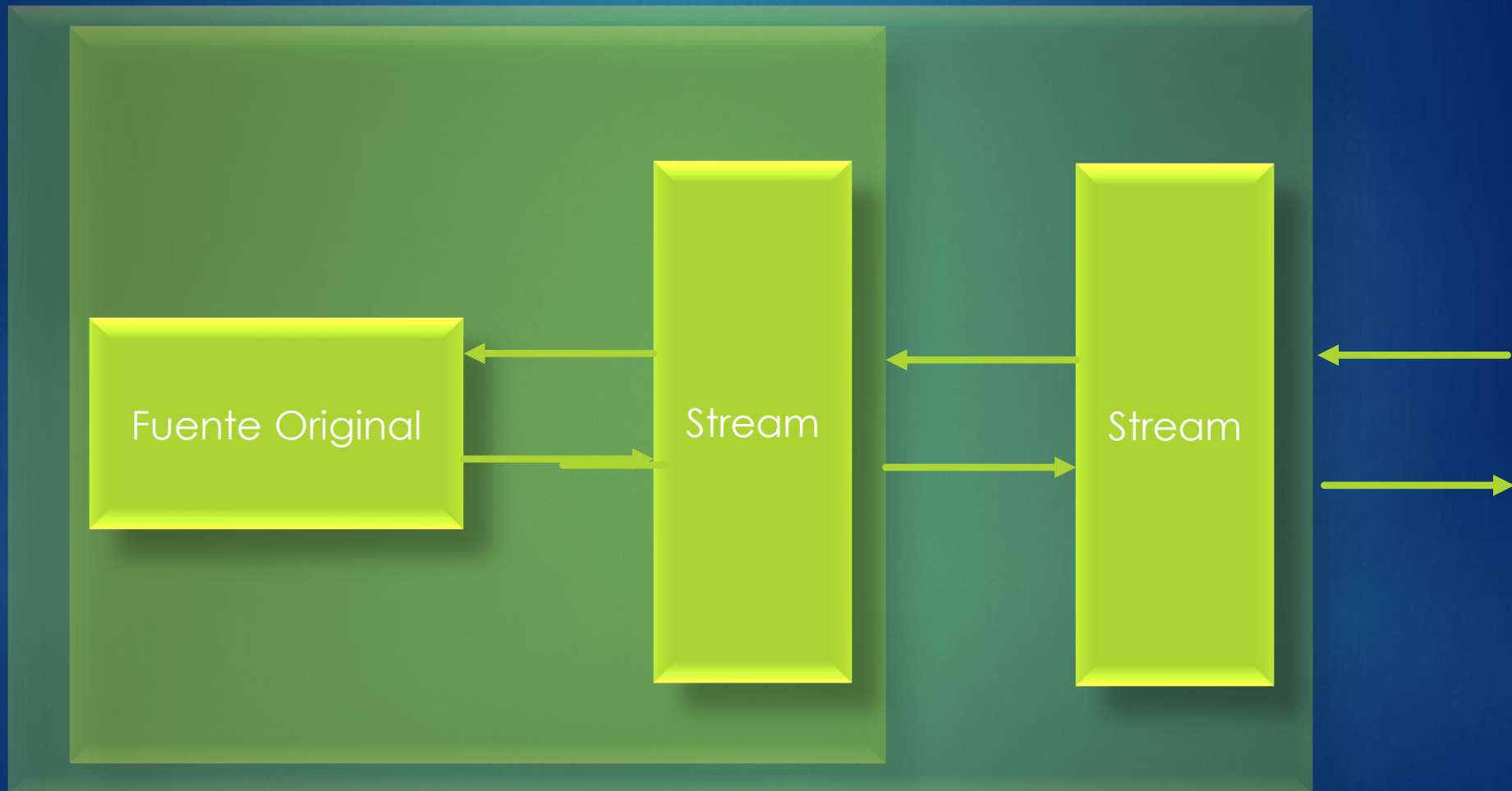
33

- ▶ Los combinadores de colección operan sobre streams no sobre la colección directamente
- ▶ Un stream es una abstracción de un cierto iterador que es perezoso
- ▶ Para usar un combinador (map, filter, ...) en una lista hay que “convertirla” en un stream
- ▶ Un stream T se puede “conectar” a otro stream S :
 - ▶ S genera un elemento o grupo de elementos y T lo(s) consume
 - ▶ Si T no pide elementos S no los produce (S es perezoso, o lazy)
- ▶ Son como los pipes de Linux/Windows. O los generadores de JS o Python.

Esquema stream

34





Ejemplo

36

```
jshell> var a = Arrays.asList(1,2,3,4,5,6)
a ==> [1, 2, 3, 4, 5, 6]

jshell> var s = a.stream()
s ==> java.util.stream.ReferencePipeline$Head@4cdf35a9

jshell> var even = s.filter(e -> e % 2 == 0)
even ==> java.util.stream.ReferencePipeline$2@6bf2d08e

jshell> var even_square = even.map(e -> e * e)
even_square ==> java.util.stream.ReferencePipeline$3@73a8dfcc

jshell> even_square.toList()
$28 ==> [4, 16, 36]

jshell> |
```

Más directo

37

► “One liner”

```
jshell> a.stream().filter( e -> e % 2 == 0).map(e -> e * 2).toList()  
$30 ==> [4, 8, 12]  
  
jshell> |
```

- ▶ Una lambda puede ver datos definidos fuera de la misma
- ▶ La clausura **léxica** consiste en cerrar la lambda para que esos datos queden atados a los valores en el lugar donde se definió
- ▶ La lambda recuerda esos valores de cuando fue definida.

Ejemplo: Clausura

39

```
jshell> int num666 = 666
num666 ==> 666

jshell> Function<Integer, Integer> add = x -> x + num666
add ==> $Lambda$38/0x00000000800c16b68@1c4af82c

jshell> void useClosure(Function<Integer, Integer> lambda){
...>     int num666 = 777;
...>     System.out.println(lambda.apply(10));
...> }
| created method useClosure(Function<Integer, Integer>)

jshell> useClosure(add)
676

jshell> |
```

Se usa en num666 de afuera
Lambda vendrá "cerrada"

add irá a useClosure
"cerrada"

Restricción Alcance

- ▶ Las variables de **cierre no pueden ser reasignadas** en una lambda si son de pila
- ▶ Debe ser como si fueran constante (final o nunca reasignable)

```
jshell> public Supplier<Integer> getLambda(){
...>         int num666 = 666;
...>         return () -> {
...>             num666 += 666;
...>             return num666;
...>         };
...>     }
|
| Error:
| local variables referenced from a lambda expression must be final or effectively final
|
|         num666 += 666;
|         ^-----^
|
| Error:
| local variables referenced from a lambda expression must be final or effectively final
|
|         return num666;
|         ^-----^
|
jshell> |
```

Restricción de alcance

41

- ▶ No aplica si la variable es de heap

```
jshell> int num666 = 0
num666 ==> 0

jshell> public Supplier<Integer> getLambda(){
...>         return () -> {
...>             num666 += 666;
...>             return num666;
...>         };
...>     }
| created method getLambda()

jshell> |
```